Numerical Precision and Training Stability

pig7selene

September 5, 2026

Abstract

Large-language-model pretraining is executed with finite representations, finite accumulators, and a finite tolerance for abnormal updates. This chapter distinguishes numerical range from precision; compares FP32, FP16, and BF16; and develops the mixed-precision contracts that keep low-precision computation compatible with reliable optimization. It derives stable forms of Softmax, Log-Sum-Exp, and cross-entropy, explains loss scaling and higher-precision accumulation, and separates representational failures from genuine optimization instability. The chapter concludes with diagnostic signals and implementation invariants for a stable pretraining run.

1 Introduction

The pretraining objective in Chapter 5 is written over real-valued logits, probabilities, and losses. The optimizer in Chapter 7 is written as an update over real-valued gradients and moment tensors. A training system, however, evaluates neither expression over the real numbers. It stores tensors in finite floating-point formats, rounds intermediate results, and chooses a precision for every reduction and parameter update. These choices affect memory traffic and arithmetic throughput, but they also decide whether a finite mathematical quantity is represented faithfully, rounded coarsely, flushed to zero, or converted to an infinity.

Numerical stability is therefore not synonymous with successful optimization. A run can be numerically invalid because a finite intermediate overflows, even when its learning rate would otherwise be reasonable. It can also remain perfectly finite while diverging because the learning rate, data, initialization, or model dynamics produce unsuitable updates. The purpose of this chapter is to make that distinction operational. It builds on the stable cross-entropy path introduced in Chapter 5, the normalization and initialization analysis in Chapter 4, and the clipping and optimizer contracts in Chapter 7. It does not address distributed execution or communication.

2 Floating-Point Representation

2.1 Range and Precision

A normalized binary floating-point number has a sign, an exponent, and a significand. Abstractly, a nonzero normalized value can be written as

$$x = (-1)^s 2^{e-b}(1+f), \tag{1}$$

where s is the sign bit, e is the encoded exponent, b is an exponent bias, and f is the fractional significand. The exponent determines the broad range of magnitudes that can be represented. The significand determines the spacing between adjacent representable values at a given magnitude. These are different resources: more exponent bits extend range, whereas more fraction bits improve relative precision.

For a normal floating-point operation rounded to nearest, the standard error model is often summarized as

$$\mathrm{fl}(x) = x(1+\delta), \quad |\delta| \le u, \tag{2}$$

where fl denotes the stored floating-point result and u is the unit roundoff of the destination format. The model excludes overflow, underflow, and exceptional values; it is a local approximation, not a

promise that an entire neural-network computation has small error. It nevertheless explains why many individually small rounding errors can accumulate in long reductions [1].

Range answers whether a magnitude can be represented at all. **Precision** answers how closely a representable number near that magnitude can approximate it. A format may have broad range and coarse increments, or narrow range and fine increments. Conflating the two leads directly to poor mixed-precision decisions.

2.2 FP32, FP16, and BF16

The three formats most relevant to modern training are FP32, IEEE binary16 (usually called FP16), and BF16. Table Table 1 gives their layout-level distinction. Fraction-bit counts exclude the implicit leading bit of normalized binary values. IEEE 754 specifies the binary floating-point formats, rounding behavior, and exceptional values that underlie FP32 and FP16 [2].

Format	Sign / exponent / fraction bits	Largest finite magnitude	Smallest normal magnitude	Primary consequence
FP32	$\frac{1}{8}$ $\frac{}{23}$	$\approx 3.4 \times 10^{38}$	$\approx 1.2 \times 10^{-38}$	Wide range and fine relative precision.
FP16	$\frac{1}{5}$ $\frac{}{10}$	65504	$\approx 6.1 \times 10^{-5}$	More fraction bits than BF16, but a much narrower exponent range.
BF16	$\frac{1}{8}$ $\frac{}{7}$	$\approx 3.4 \times 10^{38}$	$\approx 1.2 \times 10^{-38}$	FP32-like range with substantially coarser relative precision.

Table 1: Format properties for normalized finite values. FP16 and BF16 use the same storage width, but allocate their bits differently; subnormals and hardware handling of them are not shown.

FP16 preserves more fraction bits than BF16, so it distinguishes more nearby values at the same scale. Its five exponent bits, however, make its largest finite value only 65504. BF16 instead keeps FP32's eight exponent bits and sacrifices fraction bits. It can therefore represent roughly the same order of magnitudes as FP32 but rounds more coarsely at each order of magnitude.

This exponent-range property is why BF16 is widely preferred for large-model training when efficient hardware support is available. Activations, gradients, and logits can vary across many orders of magnitude, and BF16 greatly reduces the risk that a finite FP32-scale value overflows or underflows solely because it was stored in a 16-bit tensor. BF16 does not make computation exact: its short significand still loses small relative changes, so important reductions and optimizer state commonly use FP32. The empirical BF16 study of Kalamkar et al. identifies its FP32-like range as the central advantage over FP16-style half precision for training [3].

3 Finite-Precision Failure Modes

3.1 Rounding, Underflow, and Overflow

Rounding replaces a real result by a nearby representable value. The effect is relative to scale: a small increment can disappear when added to a much larger value because the format has no representable number between the larger value and its next neighbor. This is one reason that updating a large FP16 parameter directly with a small FP16 increment can silently produce no change.

Underflow occurs when a nonzero mathematical quantity is too small for the destination representation. A format may retain a subnormal value near zero, round the result to zero, or use a kernel policy that flushes subnormals to zero for performance. In all of these cases, a stored zero need not mean that the real-valued quantity was zero. Small gradients are especially vulnerable in FP16 because its normal range begins near 6.1×10^{-5}. Underflow is a representational event, not evidence that the corresponding derivative is mathematically absent.

Overflow occurs when a finite mathematical magnitude exceeds the largest finite value available in the destination format. Under ordinary round-to-nearest behavior it typically produces a signed infinity. Subsequent invalid expressions, such as infinity minus infinity or zero times infinity, can

produce a Not a Number (NaN). Overflow is also distinct from an exploding gradient. An exploding gradient is a model- or optimization-level statement that a derivative norm has become very large; overflow is a format-level statement that some finite quantity no longer fits. A large gradient may overflow in FP16 while remaining finite in FP32 or BF16, and a gradient can be genuinely explosive without yet crossing any format limit.

3.2 Forward Precision and Accumulation Precision

The precision used to store or multiply inputs is not necessarily the precision used to accumulate a result. For a dot product,

$$r = \sum_{i=1}^{n} a_i b_i, \tag{3}$$

an implementation may read a_i and b_i from BF16 or FP16 tensors, form low-precision products, and accumulate the partial sum in FP32 before rounding the output for storage. This is materially different from accumulating the entire sum in the input format. The former retains the bandwidth and matrix-multiply advantages of a 16-bit data path while reducing error in the reduction; the latter repeatedly rounds a running total that may contain thousands of products.

The same distinction applies to loss sums, normalization statistics, gradient norms, and optimizer moments. In this chapter, **forward precision** means the format in which a tensor participates in the main forward and backward arithmetic, whereas **accumulation precision** means the format of partial sums, reductions, or update state. A mixed-precision policy must name both. Saying only that a run uses "BF16" leaves the behavior of its reductions unspecified.

4 Mixed-Precision Training

Mixed-precision training assigns different numerical roles to different tensors and operations. A common policy stores activations, model-facing weights, and many gradients in a 16-bit format; performs selected reductions and matrix accumulations in FP32; and preserves an FP32 parameter representation or optimizer state for updates. This separation reduces memory and often accelerates compute without asking every operation to tolerate the same error budget. The original mixed-precision recipe explicitly combines low-precision model tensors with FP32 master weights, higher-precision accumulation, and loss scaling for FP16 gradients [4].

4.1 Master Weights and Optimizer State

Let θ^{master} denote a higher-precision parameter copy and let θ^{model} be the rounded copy used by forward and backward kernels. A simplified update path is

$$\theta_{t+1}^{\text{master}} = \text{Update}(\theta_t^{\text{master}}, g_t), \quad \theta_{t+1}^{\text{model}} = \text{cast}(\theta_{t+1}^{\text{master}}). \tag{4}$$

The master copy protects small updates that would be lost if a low-precision weight were updated in place. The cast model copy may still be coarse, but the optimizer does not repeatedly discard sub-unit updates from the parameter trajectory. Chapter 7 explains the AdamW update itself; the numerical point here is that its first and second moment tensors, its denominator, and its parameter update should be maintained in a precision compatible with their dynamic range and required resolution.

An FP32 master copy is particularly important in the classical FP16 recipe. BF16's wider range reduces overflow and gradient-underflow pressure, but it does not provide FP32-level parameter resolution. Frameworks therefore often retain FP32 optimizer states and may retain FP32 master weights even when BF16 is the model-facing format. This is a configuration decision, not an invariant of the name BF16: the checkpoint must record which copy is authoritative and which tensors are merely compute casts.

4.2 Gradient Scaling and Loss Scaling

In FP16, many mathematically valid gradients can be too small to survive the backward pass. Loss scaling multiplies the scalar loss by a positive scale S before differentiation:

$$\mathcal{L}_S = S\mathcal{L}, \quad g_S = \nabla_\theta \mathcal{L}_S = Sg. \tag{5}$$

Before the optimizer operates, the implementation divides the gradient by the same scale,

$$g = \frac{g_S}{S}. \tag{6}$$

After this unscaling, the intended objective and optimizer update are unchanged, apart from finite-precision rounding. Loss scaling does not redefine maximum likelihood, increase the learning rate, or create additional signal; it shifts intermediate gradient magnitudes into a format's representable range and then reverses the shift. The loss-scaling method is useful only when the unscaling occurs before operations whose thresholds have semantic meaning, such as gradient clipping, optimizer updates, and gradient-norm logging. Clipping g_S with the ordinary threshold c, for example, is not equivalent to clipping g with c.

Dynamic loss scaling adjusts S during training. A typical policy detects nonfinite scaled gradients, skips the affected optimizer update, lowers S, and retries future batches at the lower scale. After a sustained interval of finite updates it may increase S cautiously. The finite-value check is part of numerical control, not an optimization step. BF16 often avoids the need for loss scaling because its exponent range matches FP32′s, but this does not eliminate the need to check for NaNs, infinities, or inaccurate low-precision reductions.

5 Stable Exponentials and Cross-Entropy

5.1 Log-Sum-Exp and Softmax

Chapter 5 defines a vocabulary logit vector $O_{b,t,:}$ and its Softmax distribution. For this section, write the logits at one supervised position as $o_1, \ldots, o_V$ and let

$$c = \max_{1 \le j \le V} o_j. \tag{7}$$

The Log-Sum-Exp function has the algebraically equivalent shifted form

$$\mathrm{LSE}(o) = \log \sum_{j=1}^{V} \exp\big(o_j\big) = c + \log \sum_{j=1}^{V} \exp\big(o_j - c\big). \tag{8}$$

Because $o_j - c \le 0$, every exponential in the right-hand sum lies in $(0, 1]$ over the real numbers. The largest term is exactly one, so a very large positive logit cannot overflow merely because it is exponentiated. Softmax uses the same shift,

$$\operatorname{softmax}(o)_v = \frac{\exp(o_v - c)}{\sum_{j=1}^{V} \exp\big(o_j - c\big)}. \tag{9}$$

The shift changes neither the exact probability distribution nor the exact Log-Sum-Exp value; it subtracts the same constant from every logit. It changes the floating-point computation decisively. Unshifted $\exp(o_v)$ can overflow for a large positive logit, while shifted exponentials do not. Conversely, a logit far below c can produce an exponential that underflows to zero. That output is a numerical zero, not a claim that the real exponential is zero. Its contribution to the denominator is often below the working precision and therefore harmless, but it should not be confused with a mathematical deletion of the token. Numerical analyses of shifted Log-Sum-Exp and Softmax formalize why these algebraically equivalent expressions behave differently in finite precision [5].

5.2 A Stable Cross-Entropy Path

For target token index y, the token-level negative log-likelihood can be written directly from logits as

$$\ell = \mathrm{LSE}(o) - o_y = c + \log \sum_{j=1}^{V} \exp\big(o_j - c\big) - o_y. \tag{10}$$

This is the numerical form that a cross-entropy kernel should implement. Computing p_y from a separately materialized Softmax vector and then evaluating $-\log p_y$ is less robust. If o_y is much smaller

than the maximum logit, p_y can underflow to stored zero even though Equation 10 remains a finite loss. The stable logit-space expression also avoids constructing a length-V probability tensor solely to gather one target probability.

The max subtraction in Equation 8 does not solve every numerical problem. The sum remains a reduction, logits may already contain infinities or NaNs, and a low-precision output may still round. Its role is narrower and essential: it prevents avoidable overflow from the exponential while preserving the mathematical objective introduced in Chapter 5.

6 Reductions, Norms, and Statistics

Reductions are numerically sensitive because they combine many values into one. Under the standard model in Equation 2, a sequential summation of n terms has a representative error bound

$$|\hat{s} - s| \le \gamma_{n-1} \sum_{i=1}^{n} |x_i|, \quad \gamma_k = \frac{ku}{1 - ku}, \tag{11}$$

when $ku < 1$, where $s = \sum_i x_i$ and $\hat{s}$ is the computed sum. The bound is not a prediction for one particular kernel, but it captures the two relevant facts: rounding error grows with the number of additions, and cancellation makes a small final sum difficult to compute relative to the magnitude of its inputs [1]. Tree reductions change the order of additions and therefore can change low-order bits even when they are mathematically equivalent.

This applies directly to the valid-token loss sum in Chapter 5 and the accumulated gradient in Chapter 7. Accumulate loss numerators, gradient statistics, and large dot products in higher precision where the kernel permits. The denominator for a masked loss must be counted under the same mask, but it is an integer accounting quantity rather than a low-precision activation reduction.

LayerNorm and RMSNorm add a further case: they estimate a scale from many feature values. Chapter 4 notes that the variance identity $E[x^2] - E[x]^2$ can suffer cancellation when the mean is large. Higher-precision accumulation and a stable variance algorithm reduce this risk. The normalizer's ε prevents division by a zero or extremely small estimated scale, but it cannot repair an activation distribution that has already become pathological.

Global gradient norms require the same care. The expression $\|g\|_2 = \sqrt{\sum_i g_i^2}$ squares each component before summing, so a finite component can overflow during squaring in a narrow format. Compute norms and clipping decisions from unscaled gradients in an appropriately wide accumulation precision. This preserves the meaning of the clipping threshold from Chapter 7 rather than making it a hidden function of the compute dtype.

7 Stability as a Training-System Property

7.1 Initialization and Scale

Numerical failures often expose an earlier scale problem. Chapter 4 derived how fan-aware initialization and residual scaling aim to keep activation and gradient variance in a useful range. Those arguments become more—not less—important in low precision. A poorly scaled projection can create activation outliers; outliers widen logit and gradient distributions; wide distributions increase the chance of overflow, loss-scale backoff, or inaccurate reductions.

Monitoring should therefore observe distributions, not only scalar loss. Useful per-block measurements include activation RMS, maximum absolute activation, high quantiles, logit range, gradient RMS, and global gradient norm. A single extreme layer can be invisible in a batch-averaged loss until it contaminates a reduction or parameter update. In contrast, a broadly rising activation scale across many layers may point to residual accumulation, normalization, or learning-rate behavior. These measurements do not identify a cause by themselves, but they localize the first boundary at which the run stopped resembling its finite baseline.

7.2 Numerical and Optimization Instability

Numerical instability is a failure of the implemented arithmetic to represent or evaluate the intended computation reliably. Its immediate signals include NaN or Inf tensors, a sudden loss-scale reduction, nonfinite optimizer moments, or a loss that changes drastically when only the compute dtype is widened. Optimization instability is a failure of the update trajectory: the loss may spike, gradients may grow, or the model may fail to improve while every tensor remains finite. It can be caused by an unsuitable learning-rate transition, a data anomaly, a batch-size change, or an architectural scale problem.

The two failures interact but should not be diagnosed as one. An optimization-induced gradient spike can trigger FP16 overflow. Conversely, repeated FP16 underflow can suppress useful gradients and create an apparent optimization plateau. Widening selected operations to FP32 is a useful isolation experiment: if a failure disappears while the data, parameters, and update schedule are held fixed, a numerical pathway is implicated; if it persists with finite FP32 tensors, the cause is more likely in the optimization or data path. Neither outcome is a proof, so logs and reproducible failing batches remain necessary.

8 Practical Stability Diagnostics

Table Table 2 summarizes common failure signals. The right response is to record the earliest abnormal tensor and isolate one variable at a time, not to apply a larger clipping threshold or a smaller learning rate indiscriminately.

Observed signal	Possible interpretation	First isolation step
NaN or Inf after one operation	Overflow, invalid arithmetic, corrupt input, or a nonfinite value entering the operation	Locate the first nonfinite tensor; rerun the same batch with selected operations widened.
Sudden finite loss spike	Data anomaly, learning-rate transition, abnormal logits, or emerging optimization instability	Compare the batch, learning-rate state, logit range, and per-layer norms with the last finite update.
Abnormal gradient norm	True gradient growth, incorrect loss reduction, missed unscaling, or low-precision norm accumulation	Verify unscaling and masks, then inspect layerwise gradient statistics in wider precision.
Activation or logit outlier	Initialization, residual-scale, normalization, data, or kernel-path issue	Identify the first affected block and compare its inputs, normalizer statistics, and residual update.
Nonfinite optimizer state	A prior nonfinite gradient or invalid update has entered the state tensors	Skip the update, preserve the failing state, and resume only from a known finite checkpoint after diagnosis.

Table 2: Diagnostic signals are evidence, not diagnoses. The goal is to identify the earliest abnormal boundary while preserving the exact batch and training state that produced it.

Sudden loss spikes deserve particular restraint. A spike can be a transient consequence of stochastic mini-batch sampling, a corrupted or unusual sequence, a learning-rate boundary, a numerical overflow that has not yet surfaced as NaN, or a genuine model instability. The decisive question is not whether the scalar loss is large, but whether the surrounding tensors, masks, learning-rate state, and optimizer state remain finite and consistent with the declared contracts. A stable system logs enough information to answer that question after the fact.

9 Implementation Contracts

A precision policy must be explicit and checkpointed. It should declare the model-facing dtype, the accumulation dtype for matrix products and reductions, the authoritative parameter dtype, the dtype of optimizer moments, and the cast boundaries between them. A configuration described only as “mixed precision” is underspecified. The policy should also state whether subnormal flushing, fused kernels, or dynamic loss scaling are active when those choices can affect reproducibility.

The loss contract follows Chapter 5: consume raw logits, evaluate cross-entropy through a stable fused Log-Sum-Exp path, and reduce only valid target positions. In an FP16 loss-scaling path, test scaled gradients for finiteness, then unscale successful gradients before computing gradient norms, applying clipping, logging optimizer-facing statistics, or updating parameters. The valid-token loss numerator, denominator, and accumulation convention must remain identical to those used by Chapter 7.

The finite-state contract checks parameters, gradients, optimizer moments, loss, and selected activation statistics at declared boundaries. A nonfinite gradient must not be allowed to update a master parameter or moment tensor. Preserve the failing batch identifiers, random state, optimizer state, precision policy, and recent scalar traces before skipping an update or restoring a checkpoint. Finally, a reference test should compare a small deterministic batch against a higher-precision implementation for logits, loss, selected gradients, and one optimizer update. This is not a requirement for bitwise equality; it is a contract that low-precision deviations are bounded, expected, and not silently changing the computation.

10 Summary

Floating-point formats trade range against precision. FP16 has a narrower exponent range despite more fraction bits than BF16; BF16 preserves FP32-like range but has coarser relative resolution. This makes BF16 a strong default for modern large-model compute when hardware supports it, while FP32 remains valuable for reductions, master parameters, and optimizer state.

Mixed-precision training works because not every operation needs the same representation. Higher-precision accumulation protects large reductions, master weights retain small updates, and loss scaling shifts FP16 gradients into range before being reversed prior to the optimizer. Stable Log-Sum-Exp and max-shifted Softmax preserve the cross-entropy objective while avoiding preventable exponential overflow. Reliable training then requires more than finite loss: it requires explicit dtype boundaries, correct unscaling and clipping order, monitored activation and gradient statistics, nonfinite-value containment, and a disciplined separation between numerical failure and optimization instability.

References

[1] N. J. Higham, *Accuracy and Stability of Numerical Algorithms*, 2nd ed. Society for Industrial, Applied Mathematics, 2002. doi: 10.1137/1.9780898718027.

[2] IEEE, "IEEE Standard for Floating-Point Arithmetic," technical report IEEE Std 754–2019, 2019. doi: 10.1109/IEEESTD.2019.8766229.

[3] D. Kalamkar *et al.*, "A Study of BFLOAT16 for Deep Learning Training," *arXiv preprint arXiv:1905.12322*, 2019, doi: 10.48550/arXiv.1905.12322.

[4] P. Micikevicius *et al.*, "Mixed Precision Training," in *International Conference on Learning Representations*, 2018. [Online]. Available: https://openreview.net/forum?id=r1gs9JgRZ

[5] P. Blanchard, D. J. Higham, and N. J. Higham, "Accurately Computing the Log-Sum-Exp and Softmax Functions," *IMA Journal of Numerical Analysis*, vol. 41, no. 4, pp. 2311–2330, 2020, doi: 10.1093/imanum/draa038.